

```

# -*- coding: utf-8 -*-
"""ラジオコント漫才〜RAKOMAN〜 : ラジオコント・ラジオ漫才専用の音声投稿サイト"""
import os
import re
import sqlite3
import uuid
import datetime
import functools

from flask import (
    Flask, g, render_template, request, redirect, url_for,
    session, send_from_directory, abort, jsonify, flash
)
from werkzeug.security import generate_password_hash, check_password_hash

# ----- 設定
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.environ.get("DATA_DIR", os.path.join(BASE_DIR, "data"))
UPLOAD_DIR = os.path.join(DATA_DIR, "uploads")
DB_PATH = os.path.join(DATA_DIR, "rakoman.db")
ADMIN_USERNAME = os.environ.get("ADMIN_USERNAME", "admin") # この名前のユーザーが管理

os.makedirs(UPLOAD_DIR, exist_ok=True)

app = Flask(__name__)
app.secret_key = os.environ.get("SECRET_KEY", "rakoman-dev-secret-change-me")
app.config["MAX_CONTENT_LENGTH"] = 80 * 1024 * 1024 # 80MB(音声+写真4枚)

# ---- 全テンプレート同梱(スマホだけでデプロイできる1ファイル版) ----
from jinja2 import DictLoader
TEMPLATES = {'profile.html': '{% extends "base.html" %}\n{% block title %}{{ user
app.jinja_loader = DictLoader(TEMPLATES)

AUDIO_EXT = {"mp3", "m4a", "wav", "aac", "ogg", "oga", "webm", "flac", "caf", "am
IMAGE_EXT = {"jpg", "jpeg", "png", "gif", "webp", "heic", "heif"}

PERIODS = [
    ("24h", "24時間", 1),
    ("1w", "1週間", 7),
    ("1m", "1ヶ月", 30),
    ("3m", "3ヶ月", 92),
    ("6m", "半年", 183),
    ("1y", "1年", 366),

```

```

        ("all", "全期間", None),
    ]
    SORTS = [("new", "新着順"), ("likes", "いいね順"), ("favs", "お気に入り順")]

# 絵文字判定 (主要な絵文字ブロックをカバー)
EMOJI_RE = re.compile(
    "[
        \U0001F300-\U0001F5FF"
        \U0001F600-\U0001F64F"
        \U0001F680-\U0001F6FF"
        \U0001F700-\U0001F77F"
        \U0001F780-\U0001F7FF"
        \U0001F900-\U0001F9FF"
        \U0001FA70-\U0001FAFF"
        \U0001F1E6-\U0001F1FF"
        \U00002600-\U000027BF"
        \U00002B00-\U00002BFF"
        \U00002190-\U000021FF"
        \U00002300-\U000023FF"
        \u2764\u2763\u203C\u2049\u3030\u303D\u3297\u3299"
    ]"
)

def distinct_emoji_count(text):
    """文章中の絵文字の「種類」を数える"""
    return len(set(EMOJI_RE.findall(text or "")))

# ----- DB
def get_db():
    if "db" not in g:
        g.db = sqlite3.connect(DB_PATH)
        g.db.row_factory = sqlite3.Row
        g.db.execute("PRAGMA foreign_keys = ON")
    return g.db

@app.teardown_appcontext
def close_db(exc):
    db = g.pop("db", None)
    if db is not None:
        db.close()

def init_db():
    db = sqlite3.connect(DB_PATH)

```

```
db.executescript(
    """
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT UNIQUE NOT NULL,
        password_hash TEXT NOT NULL,
        icon TEXT,
        created_at TEXT NOT NULL
    );
    CREATE TABLE IF NOT EXISTS posts (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
        title TEXT NOT NULL,
        description TEXT DEFAULT '',
        hashtags TEXT DEFAULT '',
        audio TEXT NOT NULL,
        views INTEGER DEFAULT 0,
        likes INTEGER DEFAULT 0,
        is_public INTEGER DEFAULT 1,
        created_at TEXT NOT NULL
    );
    CREATE TABLE IF NOT EXISTS post_images (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        post_id INTEGER NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
        filename TEXT NOT NULL,
        position INTEGER DEFAULT 0
    );
    CREATE TABLE IF NOT EXISTS favorites (
        user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
        post_id INTEGER NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
        created_at TEXT NOT NULL,
        PRIMARY KEY (user_id, post_id)
    );
    CREATE TABLE IF NOT EXISTS comments (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        post_id INTEGER NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
        user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
        body TEXT NOT NULL,
        created_at TEXT NOT NULL
    );
    CREATE TABLE IF NOT EXISTS playlists (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
        title TEXT NOT NULL,
        is_public INTEGER DEFAULT 0,
        created_at TEXT NOT NULL
    );
    );
```

```

CREATE TABLE IF NOT EXISTS playlist_items (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    playlist_id INTEGER NOT NULL REFERENCES playlists(id) ON DELETE CASCADE,
    post_id INTEGER NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
    position INTEGER DEFAULT 0,
    UNIQUE (playlist_id, post_id)
);
"""
)
# 既存DBへのマイグレーション:自己紹介カラム
cols = [r[1] for r in db.execute("PRAGMA table_info(users)").fetchall()]
if "bio" not in cols:
    db.execute("ALTER TABLE users ADD COLUMN bio TEXT DEFAULT ''")
pcols = [r[1] for r in db.execute("PRAGMA table_info(posts)").fetchall()]
if "thumb_id" not in pcols:
    db.execute("ALTER TABLE posts ADD COLUMN thumb_id INTEGER")
db.commit()
db.close()

init_db()

def now_str():
    return datetime.datetime.utcnow().strftime("%Y-%m-%d %H:%M:%S")

# ----- 認証まわり
def current_user():
    uid = session.get("user_id")
    if not uid:
        return None
    return get_db().execute("SELECT * FROM users WHERE id = ?", (uid,)).fetchone()

def is_admin(user):
    return bool(user) and user["username"] == ADMIN_USERNAME

def login_required(view):
    @functools.wraps(view)
    def wrapped(*args, **kwargs):
        if not current_user():
            flash("ログインしてください 🙏")
            return redirect(url_for("login", next=request.path))
        return view(*args, **kwargs)
    return wrapped

```

```
@app.context_processor
def inject_globals():
    user = current_user()
    return dict(me=user, me_is_admin=is_admin(user), SORTS=SORTS, PERIODS=PERIODS)
```

```
# ----- ファイル
```

```
def ext_of(filename):
    return filename.rsplit(".", 1)[-1].lower() if "." in filename else ""
```

```
def save_upload(file, allowed):
    ext = ext_of(file.filename)
    if ext not in allowed:
        return None
    name = uuid.uuid4().hex + "." + ext
    file.save(os.path.join(UPLOAD_DIR, name))
    return name
```

```
def delete_file(name):
    if not name:
        return
    try:
        os.remove(os.path.join(UPLOAD_DIR, name))
    except OSError:
        pass
```

```
@app.route("/media/<path:filename>")
def media(filename):
    return send_from_directory(UPLOAD_DIR, filename, conditional=True)
```

```
# ----- 投稿一覧クエリ
```

```
POST_SELECT = """
SELECT p.*, u.username, u.icon AS user_icon,
       (SELECT COUNT(*) FROM favorites f WHERE f.post_id = p.id) AS fav_count,
       (SELECT COUNT(*) FROM comments c WHERE c.post_id = p.id) AS comment_count,
       COALESCE(
           (SELECT filename FROM post_images i WHERE i.id = p.thumb_id AND i.post_id = p.id)
           (SELECT filename FROM post_images i WHERE i.post_id = p.id ORDER BY pos_i
            ) AS thumb
       ) AS thumb
FROM posts p JOIN users u ON u.id = p.user_id
"""
```

```

def fetch_posts(where="", params=(), sort="new", period="all"):
    conds = [where] if where else []
    args = list(params)
    days = next((d for key, _, d in PERIODS if key == period), None)
    if days:
        since = (datetime.datetime.utcnow() - datetime.timedelta(days=days)).strf
        conds.append("p.created_at >= ?")
        args.append(since)
    sql = POST_SELECT
    if conds:
        sql += " WHERE " + " AND ".join(conds)
    order = {"new": "p.created_at DESC, p.id DESC",
            "likes": "p.likes DESC, p.created_at DESC, p.id DESC",
            "favs": "fav_count DESC, p.created_at DESC, p.id DESC"}.get(sort, "p
    sql += " ORDER BY " + order + " LIMIT 200"
    return get_db().execute(sql, args).fetchall()

```

```

def split_tags(hashtags):
    return [t for t in re.split(r"[\s,、 ]+", (hashtags or "").replace("#", " ")) :

```

```

app.jinja_env.filters["tags"] = split_tags

```

```

# ----- ページ
@app.route("/")
def index():
    sort = request.args.get("sort", "new")
    period = request.args.get("period", "all")
    q = (request.args.get("q") or "").strip()
    where, params = "p.is_public = 1", []
    if q:
        like = f"%{q}%"
        where += " AND (p.title LIKE ? OR u.username LIKE ? OR p.hashtags LIKE ?)
        params = [like, like, like.replace("#", "")]
    posts = fetch_posts(where, params, sort, period)
    return render_template("index.html", posts=posts, sort=sort, period=period, q

```

```

# ----- 会員登録 / ログイン
@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "POST":
        username = (request.form.get("username") or "").strip()

```

```

password = request.form.get("password") or ""
if not re.fullmatch(r"[0-9A-Za-zあ-んア-ケー-齋ー_.・]{1,20}", username):
    flash("ユーザーネームは1~20文字で、使えない記号が含まれています 🙅")
elif len(password) < 6:
    flash("パスワードは6文字以上にしてください 🗝")
else:
    db = get_db()
    try:
        db.execute(
            "INSERT INTO users (username, password_hash, created_at) VALUES (username, generate_password_hash(password), now_str()),"
        )
        db.commit()
    except sqlite3.IntegrityError:
        flash("そのユーザーネームはすでに使われています 😓")
    else:
        user = db.execute("SELECT * FROM users WHERE username = ?", (username,))
        session["user_id"] = user["id"]
        flash(f"ようこそ {username} さん! 🎉")
        return redirect(url_for("index"))
return render_template("register.html")

```

```
@app.route("/login", methods=["GET", "POST"])
```

```
def login():
```

```

    if request.method == "POST":
        username = (request.form.get("username") or "").strip()
        password = request.form.get("password") or ""
        user = get_db().execute("SELECT * FROM users WHERE username = ?", (username,))
        if user and check_password_hash(user["password_hash"], password):
            session["user_id"] = user["id"]
            flash("おかえりなさい! 🏠")
            return redirect(request.args.get("next") or url_for("index"))
        flash("ユーザーネームかパスワードが違います 😞")
    return render_template("login.html")

```

```
@app.route("/logout")
```

```
def logout():
```

```

    session.clear()
    flash("ログアウトしました 🙌")
    return redirect(url_for("index"))

```

```
# ----- 投稿
```

```
@app.route("/upload", methods=["GET", "POST"])
```

```
@login_required
```

```

def upload():
    if request.method == "POST":
        me = current_user()
        title = (request.form.get("title") or "").strip()
        description = (request.form.get("description") or "").strip()
        hashtags = " ".join(split_tags(request.form.get("hashtags")))
        is_public = 1 if request.form.get("is_public") == "1" else 0
        audio = request.files.get("audio")
        if not title:
            flash("タイトルを入れてください 📌")
        elif not audio or not audio.filename:
            flash("音声ファイルを選んでください 🎧 ")
        else:
            audio_name = save_upload(audio, AUDIO_EXT)
            if not audio_name:
                flash("対応していない音声形式です (mp3 / m4a / wav などをお願いします) 🙏")
            else:
                db = get_db()
                cur = db.execute(
                    "INSERT INTO posts (user_id, title, description, hashtags, au"
                    " VALUES (?, ?, ?, ?, ?, ?, ?)",
                    (me["id"], title, description, hashtags, audio_name, is_public, )
                )
                post_id = cur.lastrowid
                images = [f for f in request.files.getlist("images") if f and f.filename]
                for i, img in enumerate(images):
                    img_name = save_upload(img, IMAGE_EXT)
                    if img_name:
                        db.execute(
                            "INSERT INTO post_images (post_id, filename, position"
                            "(post_id, img_name, i),
                        )
                db.commit()
                flash("投稿しました! 🎉📁")
                return redirect(url_for("post_page", post_id=post_id))
    return render_template("upload.html")

def get_post_or_404(post_id):
    post = get_db().execute(POST_SELECT + " WHERE p.id = ?", (post_id,)).fetchone()
    if not post:
        abort(404)
    return post

def can_view(post, user):
    return post["is_public"] or (user and (user["id"] == post["user_id"] or is_admin))

```



```

@app.route("/post/<int:post_id>")
def post_page(post_id):
    db = get_db()
    post = get_post_or_404(post_id)
    me = current_user()
    if not can_view(post, me):
        abort(404)
    db.execute("UPDATE posts SET views = views + 1 WHERE id = ?", (post_id,))
    db.commit()
    images = db.execute(
        "SELECT * FROM post_images WHERE post_id = ? ORDER BY position", (post_id,
    ).fetchall()
    comments = db.execute(
        "SELECT c.*, u.username, u.icon AS user_icon FROM comments c JOIN users u
        " WHERE c.post_id = ? ORDER BY c.created_at",
        (post_id,)),
    ).fetchall()
    faved = False
    my_playlists = []
    if me:
        faved = db.execute(
            "SELECT 1 FROM favorites WHERE user_id = ? AND post_id = ?", (me["id"]
        ).fetchone() is not None
        my_playlists = db.execute(
            "SELECT pl.*, (SELECT 1 FROM playlist_items pi WHERE pi.playlist_id =
            " FROM playlists pl WHERE pl.user_id = ? ORDER BY pl.created_at DESC"
            (post_id, me["id"])),
        ).fetchall()
    return render_template(
        "post.html", post=post, images=images, comments=comments,
        faved=faved, my_playlists=my_playlists,
    )

```

```

@app.post("/post/<int:post_id>/like")
def like(post_id):
    db = get_db()
    db.execute("UPDATE posts SET likes = likes + 1 WHERE id = ?", (post_id,))
    db.commit()
    likes = db.execute("SELECT likes FROM posts WHERE id = ?", (post_id,)).fetcho
    if not likes:
        abort(404)
    return jsonify(likes=likes["likes"])

```

```

@app.post("/post/<int:post_id>/favorite")
@login_required
def favorite(post_id):
    me = current_user()
    db = get_db()
    get_post_or_404(post_id)
    row = db.execute(
        "SELECT 1 FROM favorites WHERE user_id = ? AND post_id = ?", (me["id"], p
    ).fetchone()
    if row:
        db.execute("DELETE FROM favorites WHERE user_id = ? AND post_id = ?", (me
            faved = False
    else:
        db.execute(
            "INSERT INTO favorites (user_id, post_id, created_at) VALUES (?, ?, ?
            (me["id"], post_id, now_str()),
        )
        faved = True
    db.commit()
    count = db.execute("SELECT COUNT(*) AS c FROM favorites WHERE post_id = ?", (
    return jsonify(faved=faved, count=count)

```

```

@app.post("/post/<int:post_id>/toggle_public")
@login_required
def toggle_public(post_id):
    me = current_user()
    post = get_post_or_404(post_id)
    if post["user_id"] != me["id"] and not is_admin(me):
        abort(403)
    db = get_db()
    db.execute("UPDATE posts SET is_public = 1 - is_public WHERE id = ?", (post_i
    db.commit()
    flash("公開設定を変更しました 🔄")
    return redirect(url_for("post_page", post_id=post_id))

```

```

@app.post("/post/<int:post_id>/delete")
@login_required
def delete_post(post_id):
    me = current_user()
    post = get_post_or_404(post_id)
    if post["user_id"] != me["id"] and not is_admin(me):
        abort(403)
    db = get_db()
    delete_file(post["audio"])
    for img in db.execute("SELECT filename FROM post_images WHERE post_id = ?", (

```

```

        delete_file(img["filename"])
    db.execute("DELETE FROM posts WHERE id = ?", (post_id,))
    db.commit()
    flash("投稿を削除しました 🗑️")
    return redirect(url_for("profile", username=post["username"]))

@app.post("/post/<int:post_id>/thumbnail")
@login_required
def set_thumbnail(post_id):
    me = current_user()
    post = get_post_or_404(post_id)
    if post["user_id"] != me["id"] and not is_admin(me):
        abort(403)
    image_id = request.form.get("image_id")
    db = get_db()
    img = db.execute(
        "SELECT id FROM post_images WHERE id = ? AND post_id = ?", (image_id, post_id)
    ).fetchone()
    if img:
        db.execute("UPDATE posts SET thumb_id = ? WHERE id = ?", (img["id"], post_id))
        db.commit()
        flash("サムネイルを設定しました! 🖼️✨")
    return redirect(url_for("post_page", post_id=post_id))

def rows_for_context(args):
    """連続再生のコンテキスト(どの一覧から再生したか)に応じた投稿リスト"""
    src = args.get("src", "feed")
    me = current_user()
    db = get_db()
    if src in ("user", "ufav"):
        u = db.execute("SELECT * FROM users WHERE username = ?", (args.get("u", " ")
        if not u:
            return []
        if src == "user":
            mine = me and (me["id"] == u["id"] or is_admin(me))
            where = "p.user_id = ?" if mine else "p.user_id = ? AND p.is_public = 1"
            return fetch_posts(where, (u["id"],), "new", "all")
        return fetch_posts(
            "p.is_public = 1 AND p.id IN (SELECT post_id FROM favorites WHERE user_id = ?)",
            (u["id"],), "new", "all",
        )
    # feed: トップの並び(新着/いいね/お気に入り)・期間・検索をそのまま引き継ぐ
    sort = args.get("sort", "new")
    period = args.get("period", "all")
    q = (args.get("q") or "").strip()

```

```

where, params = "p.is_public = 1", []
if q:
    like = f"%{q}%"
    where += " AND (p.title LIKE ? OR u.username LIKE ? OR p.hashtags LIKE ?)"
    params = [like, like, like.replace("#", "")]
return fetch_posts(where, params, sort, period)

@app.get("/post/<int:post_id>/next")
def next_post(post_id):
    rows = rows_for_context(request.args)
    ids = [r["id"] for r in rows]
    if post_id in ids:
        i = ids.index(post_id) + 1
        if i < len(rows):
            nxt = rows[i]
            db = get_db()
            db.execute("UPDATE posts SET views = views + 1 WHERE id = ?", (nxt["id"],))
            db.commit()
            qs = request.query_string.decode()
            return jsonify(
                end=False,
                id=nxt["id"],
                title=nxt["title"],
                artist=nxt["username"],
                audio=url_for("media", filename=nxt["audio"]),
                artwork=url_for("media", filename=nxt["thumb"]) if nxt["thumb"] else None,
                post_url=url_for("post_page", post_id=nxt["id"]) + ("?" + qs if q else ""),
                next_endpoint=url_for("next_post", post_id=nxt["id"]) + ("?" + qs if q else "")
            )
    return jsonify(end=True)

# ----- コメント
@app.post("/post/<int:post_id>/comment")
@login_required
def add_comment(post_id):
    me = current_user()
    get_post_or_404(post_id)
    body = (request.form.get("body") or "").strip()
    if not body:
        flash("コメントを入力してください 🙅")
    elif distinct_emoji_count(body) < 3:
        flash("コメントには絵文字を3種類以上入れてください! 😞🎉👏👏")
    else:
        db = get_db()
        db.execute(

```

```

        "INSERT INTO comments (post_id, user_id, body, created_at) VALUES (?,
        (post_id, me["id"], body, now_str()),
    )
    db.commit()
    flash("コメントしました!💬✨")
    return redirect(url_for("post_page", post_id=post_id) + "#comments")

```

```

@app.post("/comment/<int:comment_id>/delete")
@login_required
def delete_comment(comment_id):
    me = current_user()
    db = get_db()
    c = db.execute(
        "SELECT c.*, p.user_id AS post_owner FROM comments c JOIN posts p ON p.id
        (comment_id,)",
    ).fetchone()
    if not c:
        abort(404)
    if c["user_id"] != me["id"] and c["post_owner"] != me["id"] and not is_admin():
        abort(403)
    db.execute("DELETE FROM comments WHERE id = ?", (comment_id,))
    db.commit()
    flash("コメントを削除しました 🗑️")
    return redirect(url_for("post_page", post_id=c["post_id"]) + "#comments")

```

----- プロフィール

```

@app.route("/user/<username>")
def profile(username):
    db = get_db()
    user = db.execute("SELECT * FROM users WHERE username = ?", (username,)).fetchone()
    if not user:
        abort(404)
    me = current_user()
    mine = bool(me) and me["id"] == user["id"]
    tab = request.args.get("tab", "posts")
    where = "p.user_id = ?" if (mine or is_admin(me)) else "p.user_id = ? AND p.is_public = 1"
    posts = fetch_posts(where, (user["id"],))
    favorites_rows = fetch_posts(
        "p.is_public = 1 AND p.id IN (SELECT post_id FROM favorites WHERE user_id = ?)"
    )
    pl_where = "user_id = ?" if mine else "user_id = ? AND is_public = 1"
    playlists = db.execute(
        f"SELECT pl.*, (SELECT COUNT(*) FROM playlist_items pi WHERE pi.playlist_id = pl.id) AS count,
        f" FROM playlists pl WHERE {pl_where} ORDER BY created_at DESC",
        (user["id"],),
    )

```

```

).fetchall()
return render_template(
    "profile.html", user=user, mine=mine, tab=tab,
    posts=posts, favorites=favorites_rows, playlists=playlists,
)

@app.post("/profile/icon")
@login_required
def change_icon():
    me = current_user()
    icon = request.files.get("icon")
    if icon and icon.filename:
        name = save_upload(icon, IMAGE_EXT)
        if name:
            delete_file(me["icon"])
            db = get_db()
            db.execute("UPDATE users SET icon = ? WHERE id = ?", (name, me["id"]))
            db.commit()
            flash("アイコンを変更しました! 📁✨")
        else:
            flash("画像ファイル(jpg / png など)を選んでください 🙏")
    return redirect(url_for("profile", username=me["username"]))

@app.post("/profile/bio")
@login_required
def change_bio():
    me = current_user()
    bio = (request.form.get("bio") or "").strip()
    if len(bio) > 500:
        flash("自己紹介は500字までです 📝(いま" + str(len(bio)) + "字)")
    else:
        db = get_db()
        db.execute("UPDATE users SET bio = ? WHERE id = ?", (bio, me["id"]))
        db.commit()
        flash("自己紹介を更新しました! 📝✨")
    return redirect(url_for("profile", username=me["username"]))

# ----- プレイリスト
@app.post("/playlists/create")
@login_required
def create_playlist():
    me = current_user()
    title = (request.form.get("title") or "").strip()
    if not title:

```

```

        flash("プレイリストのタイトルを入れてください 🍷")
        return redirect(request.referrer or url_for("index"))
db = get_db()
cur = db.execute(
    "INSERT INTO playlists (user_id, title, is_public, created_at) VALUES (?,
    (me["id"], title, 1 if request.form.get("is_public") == "1" else 0, now_s
)
)
post_id = request.form.get("post_id")
if post_id:
    db.execute(
        "INSERT OR IGNORE INTO playlist_items (playlist_id, post_id) VALUES (
        (cur.lastrowid, post_id),
        )
    )
db.commit()
flash(f"プレイリスト「{title}」を作りました!🎵")
return redirect(request.referrer or url_for("playlist_page", playlist_id=cur.

def get_playlist_or_404(playlist_id):
    pl = get_db().execute(
        "SELECT pl.*, u.username FROM playlists pl JOIN users u ON u.id = pl.user
        (playlist_id,)",
    ).fetchone()
    if not pl:
        abort(404)
    return pl

@app.route("/playlist/<int:playlist_id>")
def playlist_page(playlist_id):
    pl = get_playlist_or_404(playlist_id)
    me = current_user()
    mine = bool(me) and me["id"] == pl["user_id"]
    if not pl["is_public"] and not mine and not is_admin(me):
        abort(404)
    items = get_db().execute(
        POST_SELECT.replace("FROM posts", ", pi.id AS item_id FROM posts") + " JO
        " WHERE pi.playlist_id = ? AND (p.is_public = 1 OR p.user_id = ?)"
        " ORDER BY pi.position, pi.id",
        (playlist_id, me["id"] if me else -1),
    ).fetchall()
    return render_template("playlist.html", pl=pl, items=items, mine=mine)

@app.post("/playlist/<int:playlist_id>/toggle_public")
@login_required
def playlist_toggle_public(playlist_id):

```

```

    pl = get_playlist_or_404(playlist_id)
    me = current_user()
    if pl["user_id"] != me["id"]:
        abort(403)
    db = get_db()
    db.execute("UPDATE playlists SET is_public = 1 - is_public WHERE id = ?", (pl
    db.commit()
    flash("プレイリストの公開設定を変更しました 🔄")
    return redirect(url_for("playlist_page", playlist_id=playlist_id))

@app.post("/playlist/<int:playlist_id>/rename")
@login_required
def playlist_rename(playlist_id):
    pl = get_playlist_or_404(playlist_id)
    me = current_user()
    title = (request.form.get("title") or "").strip()
    if pl["user_id"] != me["id"]:
        abort(403)
    if title:
        db = get_db()
        db.execute("UPDATE playlists SET title = ? WHERE id = ?", (title, playlis
        db.commit()
        flash("タイトルを変更しました ✎")
    return redirect(url_for("playlist_page", playlist_id=playlist_id))

@app.post("/playlist/<int:playlist_id>/delete")
@login_required
def playlist_delete(playlist_id):
    pl = get_playlist_or_404(playlist_id)
    me = current_user()
    if pl["user_id"] != me["id"] and not is_admin(me):
        abort(403)
    db = get_db()
    db.execute("DELETE FROM playlists WHERE id = ?", (playlist_id,))
    db.commit()
    flash("プレイリストを削除しました 🗑")
    return redirect(url_for("profile", username=me["username"], tab="playlists"))

@app.post("/playlist/<int:playlist_id>/toggle_item")
@login_required
def playlist_toggle_item(playlist_id):
    pl = get_playlist_or_404(playlist_id)
    me = current_user()
    if pl["user_id"] != me["id"]:

```



```

        abort(403)
    post_id = request.form.get("post_id")
    db = get_db()
    row = db.execute(
        "SELECT 1 FROM playlist_items WHERE playlist_id = ? AND post_id = ?", (pl
    ).fetchone()
    if row:
        db.execute("DELETE FROM playlist_items WHERE playlist_id = ? AND post_id
        flash(f"「{pl['title']}」から外しました ー")
    else:
        db.execute(
            "INSERT OR IGNORE INTO playlist_items (playlist_id, post_id) VALUES (
            (playlist_id, post_id),
        )
        flash(f"「{pl['title']}」に追加しました!♪♪")
    db.commit()
    return redirect(request.referrer or url_for("playlist_page", playlist_id=play

@app.post("/playlist/<int:playlist_id>/move")
@login_required
def playlist_move(playlist_id):
    pl = get_playlist_or_404(playlist_id)
    me = current_user()
    if pl["user_id"] != me["id"]:
        abort(403)
    item_id = int(request.form.get("item_id") or 0)
    direction = request.form.get("dir") # up / down
    db = get_db()
    items = db.execute(
        "SELECT id FROM playlist_items WHERE playlist_id = ? ORDER BY position, i
    ).fetchall()
    ids = [r["id"] for r in items]
    if item_id in ids:
        i = ids.index(item_id)
        j = i - 1 if direction == "up" else i + 1
        if 0 <= j < len(ids):
            ids[i], ids[j] = ids[j], ids[i]
            for pos, iid in enumerate(ids):
                db.execute("UPDATE playlist_items SET position = ? WHERE id = ?",
                db.commit()
    return redirect(url_for("playlist_page", playlist_id=playlist_id))

@app.errorhandler(404)
def not_found(e):
    return render_template("404.html"), 404

```

```
@app.errorhandler(413)
def too_large(e):
    flash("ファイルが大きすぎます (合計80MBまで) 🙏")
    return redirect(request.referrer or url_for("index"))

if __name__ == "__main__":
    app.run(debug=True, host="0.0.0.0", port=5000)
```